



DEPARTMENT OF
COMPUTER SCIENCE



PSEUDOCODES II

DR JEREMIAH O. BANDELE

PhD Electrical and Electronic Engineering

University of Nottingham



LEARNING OBJECTIVES

At the end of the lesson, you should be able to:

- Discuss the differences between algorithms and pseudocode.
- Outline the various steps to writing a pseudocode.
- Demonstrate how to write a pseudocode.



ALGORITHMS AND PSEUDOCODE

Algorithms are step-by-step procedures for solving a problem and it can be expressed in any language, natural language, programming code or mathematical symbol.

Pseudocode are high-level representations of an algorithm that employ a combination of natural and programming language elements; without following the syntax of a specific programming language strictly.

It is not meant to be executed as the actual code; it just clearly describes the flow and logic of an algorithm.



ALGORITHMS AND PSEUDOCODE

Algorithms can be written in any notation or language while pseudocode are written in a combination of natural and programming language elements.

Algorithms are usually represented as pseudocode during early software development stages; before they are coded in a specific programming language.

Algorithms have precise and well-defined procedures while pseudocode are more high level and informal.



WRITING A PSEUDOCODE

Consider the problem of determining the number of students that either passed or failed an examination. Also, if more than seven students pass the examination, the tuition fee will be increased.

We want to write a pseudocode that solves this problem.

To understand how to write a pseudocode, we will write a pseudocode to solve the problem stated above; by following the steps required to write a pseudocode.



WRITING A PSEUDOCODE

The steps to be followed to write a pseudocode are;

- Problem definition.
- Identification of main steps.
- Header writing.
- Using variables and data structures.
- Using constructs to describe logic.
- Using clear and concise language.
- Testing and refining.



PROBLEM DEFINITION

A clear understanding of the problem is required before we can start writing pseudocode. The inputs, process, outputs and requirements of the problem should be clearly defined.

Let us apply this step to the sample problem;

- The input will be the number of students and their examination results.



PROBLEM DEFINITION

- The process will involve accepting the examination results of each of the student and determining if the student passed or failed.
- The output will be the number of passes, the number of failures and a determination of if tuition fees should be increased or not.



IDENTIFICATION OF MAIN STEPS

This step involves the breaking down the problem into its integral parts and identifying the major steps needed to solve it.

Let us apply this step to the sample problem;

- Step 1: Initialise the storage elements.
- Step 2: Setting up all counters.
- Step 3: Accepting examination results and determining if the student passed or failed.



MID LESSON QUESTIONS

1. _____ can be written in any notation or language while
_____ are written in a combination of natural and
programming language elements.
 - a) **Algorithms...Pseudocode**
 - b) Pseudocode...Algorithms



IDENTIFICATION OF MAIN STEPS

- Step 4: Incrementing relevant counters based on the outcome of Step 3.
- Step 5: Displaying all outputs.



HEADER WRITING

For the actual implementation of the pseudocode, the first thing to do is to write a header. The header gives a description of the problem to be solved.

The header should include;

- the name of the algorithm.
- a short description of the purpose of the algorithm.
- a list of the required inputs and outputs.

Let us apply this step to the sample problem;



HEADER WRITING

- **Description:** This algorithm will accept the examination results of students and determine the number of students that passed or failed. This information will be used to determine if tuition fee should be increased or not.
- **Outputs:** Number of passes, the number of failures and a determination of if tuition fees should be increased or not.
- **Inputs:** Number of students and their results.
- **Name:** Algorithm to determine if tuition should be increased or not.



USING VARIABLES AND DATA STRUCTURE

Variables and data structures are used for data storage in a program. The name and data types of the variables should represent exactly what type of information is being stored.

Let us apply this step to the sample problem;

- A counter variable named 'passes' will be used to store the number of students that passed the examination.



USING VARIABLES AND DATA STRUCTURE

- A counter variable named 'student' will store a number indicating the current student examination result. This number will be incremented as the program receives more results.
- A counter variable named 'failures' will be used to store the number of students that failed the examination.
- All the variable names can be of an 'integer' data type.



USING CONSTRUCTS TO DESCRIBE LOGIC

To solve problems with computer programs, the flow and logic of the program should be expressed with constructs such as FOR, WHILE, CASE, SEQUENCE, REPEAT-UNTIL AND IF-THEN-ELSE.

The use of these constructs should be such that the problem solving steps are described without ambiguity.

Let us apply this step to the sample problem;



USING CONSTRUCTS TO DESCRIBE LOGIC

The WHILE construct is used to iterate over all the results one after the other.

The IF-THEN-ELSE construct is used to increment specific counters based on if some conditions are satisfied or not.

- If an examination result shows that a student passed, the passes counter is incremented.
- If an examination result shows that a student failed, the failures counter is incremented.



USING CLEAR AND CONCISE LANGUAGE

It is important to ensure that pseudocode are written in languages that are not ambiguous. Complicated syntaxes and technical terms that will make the understanding of the program logic difficult should be avoided.

```
FOR i = 0 to 20
  IF string [i] = '#'
    RETURN "Hash found"
  END IF
END FOR
```

```
FOR each index of string
  IF string (index) is equal to hash THEN
    RETURN "Hash found"
  END IF
END FOR
```

```
FOR each index of string
  If string [i] = '#'
    Return "Hash found"
  END if
END FOR
```



TESTING AND REFINING

After the pseudocode is written, it must be tested either on paper or by just scanning through to ensure that it;

- accurately describes the flow and logic of the program.
- handles all the required cases.

Refining of the pseudocode can also be carried out either to correct errors or to improve on the effectiveness of the algorithm.



SOLUTION TO PROBLEM

```
Initialise passes to zero
Initialise failures to zero
initialise student to one
WHILE student counter is active
    input the next exam result
    IF the student passed THEN
        add one to passes
    ELSE
        add one to failures
add one to student counter
print the number of passes
print the number of failures
if seven or more students passed
    print "increase tuition"
```



FURTHER READING RESOURCES

- Goodrich, M. T., & Tamassia, R. (2015). Algorithm design and applications (Vol. 363). Hoboken: Wiley.
- Shaffer, C. A. (1997). A practical introduction to data structures and algorithm analysis. Upper Saddle River, NJ: Prentice Hall.
- How to Write Pseudocode with Rules and Examples.
<https://www.prepbytes.com/blog/general/how-to-write-pseudocode-rules-and-examples/>



REFERENCES

- Goodrich, M. T., & Tamassia, R. (2015). Algorithm design and applications (Vol. 363). Hoboken: Wiley.
- Shaffer, C. A. (1997). A practical introduction to data structures and algorithm analysis. Upper Saddle River, NJ: Prentice Hall.
- How to Write Pseudocode with Rules and Examples.
<https://www.prepbytes.com/blog/general/how-to-write-pseudocode-rules-and-examples/>



DEPARTMENT OF
COMPUTER SCIENCE



THANK YOU